

Evidencia 1: Implementación de técnicas de aprendizaje máquina. (Portafolio Implementación)

Guadalupe Paulina López Cuevas A01701095@tec.mx

Resumen—

En este proyecto tiene la implementación de un sistema de detección de noticias falsas y reales basado en técnicas de deep learning. El proyecto se enfoca en la preparación del dataset, el preprocesamiento y el desarrollo de un modelo inicial de clasificación utilizando arquitecturas neuronales recurrentes. El flujo de trabajo está estructurado en tres notebooks: ETL, entrenamiento y predicción, para garantizar modularidad y reproducibilidad. El primer dataset fue obtenido de la plataforma Hugging Face, consiste en artículos etiquetados como noticias reales y falsas. Después de una limpieza exhaustiva, transformaciones, vectorización y generación del vocabulario, los datos se dividieron en conjuntos balanceados de train, validation y test. Se entrenó un modelo LSTM bidireccional con capas de regularización, alcanzando una precisión estable y demostrando ausencia de sobreajuste durante las primeras pruebas. Para la segunda parte del proyecto se implementaron mejoras como cambio de dataset obtenido de la plataforma Kaggle, para mayor número de observaciones, modificación de la arquitectura del modelo, cambio de capas de regularización y nueva tokenización y vectorización para las predicciones.

I. INTRODUCCIÓN

La detección automática de noticias falsas se ha convertido en un desafío relevante dentro del procesamiento de lenguaje natural (NLP) debido al impacto social que la desinformación genera en ámbitos políticos, económicos y culturales. Debido a esto, este proyecto desarrolla un sistema de clasificación binaria enfocado en distinguir entre noticias reales y falsas mediante técnicas basadas en deep learning.

Para esta primera etapa del proyecto, se utilizó el dataset “Fake News Detection” de Hugging Face, el cual incluye textos categorizados en dos clases. Este proyecto está organizado en tres notebooks: ETL (extracción, transformación y limpieza), ENT (entrenamiento y validación del modelo) y PRED (predicciones en datos nuevos). Esta estructura permite un flujo de trabajo modular y escalable.

Link del dataset: <https://huggingface.co/datasets/Pulk17/Fake-News-Detection-dataset>

Durante la fase ETL se preparó el dataset mediante limpieza, eliminación de columnas irrelevantes, verificación de valores nulos, reordenamiento (shuffle) de observaciones y

vectorización de texto utilizando TextVectorization de Keras. Después se definió la arquitectura inicial basada en una capa Embedding, redes LSTM bidireccionales y capas densas con regularización mediante Dropout.

Los resultados mostraron un desempeño consistente entre las métricas de entrenamiento y validación, lo que indico buena capacidad de generalización. Esta entrega constituye la base para implementar mejoras al modelo en etapas posteriores, con el objetivo de optimizar precisión, estabilidad y robustez ante texto real no estructurado.

II. INFORMACIÓN SOBRE DATASET

El dataset “Fake News Detection” es un dataset obtenido a partir de la plataforma Hugging Face en el que se encuentra una recopilación de noticias reales y falsas (en inglés).

Este dataset cuenta con 6 atributos:

Unnamed	int	ID de identificación
title	String	Título de la noticia
text	String	Párrafo
subject (sin unidad)	String	Categoría de la noticia
date	Datetime	Fecha en la que se publicó la noticia
label	Booleano	Fake news = 0 True news = 1

II. ESPECIFICACIONES SOBRE EL PROYECTO

Para este proyecto se trabajará con Notebook Colab, en este caso el proyecto estará seccionado en 3 etapas que se encontraran de igual forma divididas en 3 notebooks, que se conectaran a través de exportación e importación de archivos por medio de Google drive:

- Notebook 1 (ETL): El objetivo de este archivo es importar el dataset original, hacer limpieza y transformaciones necesarias para preparar los datos para el modelo, crear el vocabulario para el modelo y hacer la separación en train, validation y test datasets para entrenar, validar y probar el modelo.
- Notebook 2 (ENT): El objetivo de este archivo es la definición de la arquitectura del modelo, entrenamiento del modelo y validación del modelo.
- Notebook 3 (PRED): El objetivo de este archivo es importar el modelo ya entrenado y hacer predicciones

con datos de prueba, además de tener la opción de introducir texto nuevo para generar predicciones.

IV. ENTORNO PARA TRABAJAR EN NOTEBOOK ETL

Para este primer Notebook (ETL) se deben hacer algunas configuraciones con el propósito de contar con todas las librerías necesarias para la primera etapa. Sin embargo, es necesario configurar cada Notebook para poder usar funciones específicas durante la primera parte del proyecto.

```
!pip install "tensorflow-text==2.19.*"
```

Código 1. Instalación de TensorFlow

Una de las primeras cosas que se debe hacer es la instalación de TensorFlow, como se muestra en el Código 1, que es una herramienta de Google que sirve para crear, entrenar y usar modelos de inteligencia artificial y aprendizaje automático (machine learning), lo que permitirá que la computadora aprenda de datos y haga predicciones (por ejemplo, reconocer imágenes o analizar texto).

```
import pandas as pd
import numpy as np
import tensorflow as tf
import tensorflow_text as tf_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import collections
```

Código 2. Instalación de librerías

Otra de las cosas importantes, como se observa en el Código 2, es que se debe hacer es la instalación de las librerías que se usaran para la limpieza, preparación y transformación de los datos.

V. EXTRACCIÓN

Para poder utilizar un dataset que nos funcione para un modelo de técnicas de aprendizaje de maquina tenemos que hacer varios pasos antes de poner empezar a entrenar nuestro modelo, en este caso es la extracción y limpieza de dichos datos. Al extraer los datos, ya sea de un Excel, .csv, .data o cualquier tipo de archivo, los ponemos en nuestro espacio de trabajo para poder visualizarlos y poder manipularlos para nuestra limpieza del dataset. De igual forma una vez extraídos es importante conseguir la información del dataset ya que esto nos da todos los atributos dentro del dataset, la cantidad de datos que hay (que nos servirá saberlo posteriormente), si hay valores nulos, y tipo de datos que se tienen.

VI. LIMPIEZA DE DATOS

Para la limpieza del dataset se hicieron varias transformaciones para asegurar que los datos que hay son los mejores.

VI.I Null count

Primero se revisa si el dataset tiene algún valor nulo que al momento de manipular los datos provoque problemas.

VI.II Remove null

En caso de que haya valores nulos, estos se deben remover. En el caso de este dataset existieron valores nulos, así que se eliminó cualquier valor que pudiera ser nulo.

VI.III Remove columns (Unnamed, title, subject, date)

Una vez que se limpia el dataset, se deben quitar las columnas que no serán relevantes para el entrenamiento ni para las predicciones. En este caso las columnas que fueron removidas fueron las siguientes:

- Unnamed: Se remueve porque es el ID de la noticia (valor único).
- title: Se remueve porque es solamente el título de la noticia y no es necesario tenerlo si se tiene la noticia completa.
- subject: Se remueve la categoría debido a que eso no influye si la noticia es falsa o no.
- date: Se remueve porque la fecha no influye si la noticia es falsa o no, a menos de que se tenga la noticia real con la que comparar, que este no es el caso.

VI.IV Hacer shuffle de datos

Finalmente se realiza un shuffle al dataset ya que en este caso este a pesar de ser un dataset balanceado (51.6% fake news, 48.4% real news) es necesario mezclar para que el modelo no entrene a partir de algunas noticias y al hacer la pruebas sea con noticias diferentes, sino que el modelo pueda ver diferentes casos de noticias, pero no todas las noticias.

Una vez finalizada la limpieza de los datos, en este caso el dataset termina con las siguientes dimensiones:

29,984 rows x 2 columns

Ecuación. 1. Dimensiones del dataset después de limpieza

VII. TRANSFORMACIÓN DE LOS DATOS

Antes de poner el dataset dentro del modelo se debe separar el dataset en train, val y test. Esto debido a que para poder hacer un buen entrenamiento del modelo es necesario que al entrenarlo no se utilicen todos los datos del dataset para que el modelo no “aprenda” los valores y que al momento de hacer el val y el test salga que tuvo un buen rendimiento cuando simplemente se terminó ajustando a los valores de entrenamiento.

Para este dataset se hizo una división en una proporción de 70/15/15 donde habrá 20,988 muestras para entrenamiento (70%), 4,498 muestras para validación (15%) y 4,498 muestras

para prueba (15%), para que nuestro split siempre sea el mismo utilizamos una semilla de “random_state = 42”, de igual forma se define “stratify=df_shuffle[“label”]” para que al separar los 3 datasets se mantengan balanceados.

$$20,988 + 4,498 + 4,498 = 29,984$$

Ecuación. 2. Tamaño del dataset a trabajar

Una vez teniendo los datasets de entrenamiento, validación y prueba, el siguiente paso es hacer una última transformación a los datos para poder meterlos al modelo.

VII.I Tokenización y Vectorización

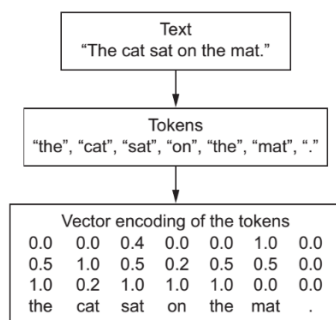


Fig. 1. Ejemplo de Tokenización y Vectorización.

La tokenización consiste en dividir en unidades llamadas tokens (palabras o subpalabras), es decir, segmentar o dividir las oraciones en fragmentos de misma “magnitud o tamaño”. La vectorización convierte estos mismos tokens en números, en este caso números enteros. Para este proceso se utilizó la capa de **TextVectorization** de Keras.

```
from tensorflow.keras.layers import TextVectorization

MAX_SEQUENCE_LENGTH = 250
VOCAB_SIZE = 20000

int_vectorize_layer = TextVectorization(
    max_tokens=VOCAB_SIZE,
    output_mode='int',
    output_sequence_length=MAX_SEQUENCE_LENGTH)
```

Código 3. Función de Tokenización y Vectorización

En el código 3, se puede ver una función que genera una matriz numérica de cada fila que representa un texto y cada columna un índice del vocabulario. Para mantener una longitud uniforme y truncar los textos que sean muy largos o cortos se definió un **MAX_SEQUENCE-LENGTH = 250**.

```
texts = dataset["text"].astype(str).values
labels = dataset["label"].values

BATCH_SIZE = 32

dataset = tf.data.Dataset.from_tensor_slices((texts,
labels)).batch(BATCH_SIZE)
```

Código 4. Separación de texto y etiquetas

En el código 4, se hace la separación del texto y las etiquetas de cada dataset para poder convertir los datasets a

tensorflow.data.Dataset y agruparlos en batches (BATCH_SIZE = 32).

Para sacar el vocabulario solo se va a utilizar el dataset del train debido a que al momento de entrenar es importante solo tomar en cuenta la información del dataset del train para que al probar el modelo en validation y test este pueda predecir sin haber visto las palabras de los nuevos datasets.

```
dataset_text = dataset.map(lambda text, label: text)
int_vectorize_layer.adapt(dataset_text)

def int_vectorize_text(text, label):
    text = tf.expand_dims(text, -1)
    return int_vectorize_layer(text), label

dataset = dataset.map(int_vectorize_text)
```

Código 5. Vectorizar y transformar datasets

Como se muestra en el Código 5, una vez teniendo los tokens vectorizados esto se mapea a cada uno de los datasets para convertir todos los tokens de los textos a los vectores (números enteros) a partir de la vectorización (vocabulario) que se generó del dataset de train.

```
AUTOTUNE = tf.data.AUTOTUNE

def configure_dataset(dataset):
    return dataset.cache().prefetch(buffer_size=AUTOTUNE)

int_dataset = configure_dataset(dataset)
```

Código 6. Configuración del dataset

Como se muestra en el Código 6, con esto se mantendrán los datos en memoria y prepara los datos del siguiente lote mientras se entrena el actual.

```
vocab = int_vectorize_layer.get_vocabulary()
```

Código 7. Vocabulario

Para poder identificar patrones en los textos es necesario tener un vocabulario, para esto el vocabulario se sacó del dataset de train donde lo que se hace es contar cuantas veces los tokens son repetidos y se acomodan de mayor frecuencia a menor frecuencia para tomar los tokens que son usados de manera más usual en fake news y en noticias reales.

VIII. IMPORTACIÓN DE DATASETS Y VOCABULARIO

Una vez teniendo los datasets preparados se exportan al igual que el vocabulario para que así se puedan importar a cualquier otro notebook que requiera esos archivos.

IX. ENTORNO PARA TRABAJAR EN NOTEBOOK ENT

Para poder empezar el entrenamiento es necesario importar los datasets de train y validation y el vocabulario.

X. PREPARACIÓN DEL DATASET DE TRAIN

Antes de crear y entrenar el modelo es necesario revisar el dataset de train y validation para saber cuáles son los valores máximos y mínimos del vocabulario y así ajustar los límites del modelo.

XI. ARQUITECTURA DEL MODELO

Creamos arquitectura para el modelo usando capas como:

- Embedding (transformación de vectores a valores que pueda ajustar el modelo),
- Bidirectional (Red neuronal recurrente para generar memoria)
- Capas densas (con funciones de activación)

Métodos de regularización como:

- Dropout (apagan neuronas de manera temporal),

```
model2 = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=input_dim,
                              output_dim=64, mask_zero=True),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64,
                                                         dropout=0.1, recurrent_dropout=0.1)),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

Código 8. Arquitectura del modelo

En el código 8, se puede ver como está estructurada la arquitectura del modelo que se utilizara tomando en cuenta que la última capa de salida debe tener una función de activación “sigmoid” debido a que este problema es de clasificación binaria.

XII. CONFIGURACIONES PARA EL MODELO DURANTE ENTRENAMIENTO

Varias de las configuraciones que se deben de hacer además de la arquitectura del modelo, son las métricas de desempeño y de ajuste y funciones regularizadoras.

```
from tensorflow.keras.callbacks import EarlyStopping,
ModelCheckpoint

model2.compile(
    loss='binary_crossentropy',
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001)
),
metrics=['accuracy']

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True,
    verbose=1
)

checkpoint = ModelCheckpoint(
    filepath=model_path,
    monitor='val_loss',
    save_best_only=True,
    save_weights_only=False,
```

```
mode='min',
verbose=1
)
```

Código 9. Inicialización de parámetros nuevos

Como se puede ver en el Código 9, aquí definimos que métricas vamos a usar para medir el desempeño del entrenamiento.

- Binary cross-entropy: que sirve para evaluar el error/perdida de las predicciones al momento del entrenamiento.
- Optimizer Adam: esto sirve para ajustar el learning rate dependiendo del desempeño del entrenamiento.
- Accuracy: esto sirve para saber cuánta precisión tuvo el modelo durante el entrenamiento.

Y definimos también funciones como:

- Early stop: detener el entrenamiento cuando no encuentra una mejora considerable en, este caso, la perdida de validation.
- Checkpoint: Guardar los pesos del modelo cuando este mejore a partir de sus métricas.

XIII. RESULTADO DE ENTRENAMIENTO

Ahora definimos el entrenamiento del modelo:

```
history = model2.fit(
    train_dataset,
    epochs=10,
    validation_data=val_dataset,
    callbacks=[early_stop, checkpoint]
)
```

Código 10. Arquitectura del modelo

Como se ve en el Código 10, para poder empezar el entrenamiento del modelo hay que hacer las siguientes configuraciones:

- Dataset con el que va a entrenar
- Numero de epochs
- Con que dataset va a validar
- Si tiene alguna función para callback (early stop, checkpoint)

```
Epoch 1/15
656/656 — val_loss improved from inf to 0.06653, saving model to /content/gdrive/myDrive/ESCUELA/IRS/7MO/IA-2/Modulo-2-2/
656/656 — 1204s 25/step - accuracy: 0.8772 - loss: 0.2578 - val_accuracy: 0.9809 - val_loss: 0.0665
Epoch 2/15
656/656 — 0s 25/step - accuracy: 0.9875 - loss: 0.0441
Epoch 2: val_loss improved from 0.06653 to 0.05220, saving model to /content/gdrive/myDrive/ESCUELA/IRS/7MO/IA-2/Modulo-
656/656 — 1182s 25/step - accuracy: 0.9875 - loss: 0.0441 - val_accuracy: 0.9833 - val_loss: 0.0522
Epoch 3/15
656/656 — 0s 25/step - accuracy: 0.9954 - loss: 0.0170
Epoch 3: val_loss did not improve from 0.05220
656/656 — 1181s 25/step - accuracy: 0.9954 - loss: 0.0170 - val_accuracy: 0.9747 - val_loss: 0.0756
Epoch 4/15
656/656 — 0s 25/step - accuracy: 0.9938 - loss: 0.0185
Epoch 4: val_loss did not improve from 0.05220
656/656 — 1191s 25/step - accuracy: 0.9938 - loss: 0.0185 - val_accuracy: 0.9804 - val_loss: 0.0734
Epoch 5/15
656/656 — 0s 25/step - accuracy: 0.9968 - loss: 0.0112
Epoch 5: val_loss did not improve from 0.05220
656/656 — 1174s 25/step - accuracy: 0.9968 - loss: 0.0112 - val_accuracy: 0.9842 - val_loss: 0.0646
Epoch 5: early stopping
Restoring model weights from the end of the best epoch: 2.
```

Fig. 2. Entrenamiento del modelo

Como se muestra en la Fig. 2, el desempeño del modelo desde el primer epoch es bastante significativo. Se puede observar que tanto el accuracy del train y de validation son bastante buenos en general. Como se ve en las primeras epochs una vez que el

modelo termina la epoch completa, si este tiene una mejora en las métricas de validation, el modelo se guarda de manera automática, para que en cada epoch se guarde la mejor versión del modelo, con el objetivo de que si llega a pasar algo que provoque que el modelo se detenga, el progreso que llevaba no se haya perdido. De igual forma, y con el objetivo haber implementado las 2 funciones regularizadoras, se puede ver que al finalizar la tercera epoch no existe una disminución de la perdida/costo del dataset de validation, es por lo que en esta tercera epoch el modelo no es guardado de manera automática al terminar la epoch. Al final se puede observar cómo después de 3 ocasiones en las que el modelo no mejoro, se optó por detener el entrenamiento y permanecer con las configuraciones que se lograron en la epoch 2.

```
val_loss, val_acc = model2.evaluate(val_dataset)

print('Test Loss:', val_loss)
print('Test Accuracy:', val_acc)
```

141/141 ————— 45s 317ms/step - accuracy: 0.9811
 Test Loss: 0.05220434069633484
 Test Accuracy: 0.9833258986473083

Fig. 3. Resultados del entrenamiento con validation

En la Fig. 3 se puede observar que, con el último modelo guardado, el dataset de validation tuvo un buen desempeño lo que significa que hasta ahorita el modelo no se sobre ajustó (overfitting) al dataset de train.

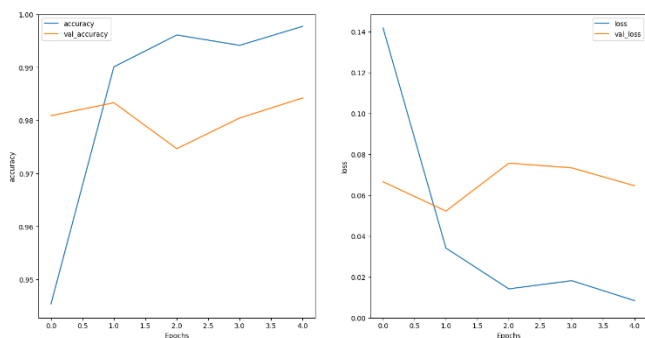


Fig. 4. Métricas Train vs Validation

En la Fig.4 se puede observar la comparación de las métricas de accuracy y loss del train y de validation, con esto podemos visualizar el desempeño del modelo y cuáles fueron los resultados de estos 2 datasets diferentes.

Ahora una vez que el modelo fue entrenado, validado y guardado ya se puede proseguir a generar predicciones nuevas y ver cómo se desempeña el modelo tanto en nuevos datasets (dataset de test) y en textos crudos.

XIV. ENTORNO PARA TRABAJAR EN NOTEBOOK PRED

Para poder empezar las predicciones es necesario importar los datasets de test y el vocabulario.

Una de las cosas que de igual forma debemos de hacer es que debido a que después de importar el vocabulario de manera automática se agregan caracteres, es necesario eliminar estos para no alterar el vocabulario original y ya después poder usar el vocabulario para transformar los textos crudos a tokens y vectorizarlos para poder usar el modelo para hacer predicciones.

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 250, 64)	1,280,000
dropout (Dropout)	(None, 250, 64)	0
bidirectional (Bidirectional)	(None, 128)	66,048
dropout_1 (Dropout)	(None, 128)	0
dense (Dense)	(None, 64)	8,256
dropout_2 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Total params: 1,354,369 (5.17 MB)
 Trainable params: 1,354,369 (5.17 MB)
 Non-trainable params: 0 (0.00 B)

Fig. 5. Modelo importado

Antes de realizar las predicciones, es bueno checar que tipo de modelo se está importando y verificar que sea el que se quiere utilizar, como se puede mostrar en la Fig. 5.

XV. PREDICCIONES PARA DATASET DE TRAIN

Como se mencionó antes, para saber si el modelo está funcionando de manera adecuado es necesario también hacer pruebas con un dataset específico para pruebas, es decir, observaciones que el modelo nunca haya visto antes. Es por esto que ahora se definen nuevamente las métricas de evaluación para ver el desempeño del modelo en datos nuevos.

```
model.compile(loss='binary_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])

test_loss, test_acc = model.evaluate(test_dataset)

print('Test Loss:', test_loss)
print('Test Accuracy:', test_acc)
```

141/141 ————— 18s 101ms/step - accuracy: 0.9849
 Test Loss: 0.052329596132040024
 Test Accuracy: 0.9831035733222961

Fig. 6. Resultados del entrenamiento con test

En la Fig. 6 se puede observar que el modelo tuvo un buen desempeño lo que nos da a entender que durante el entrenamiento el modelo si aprendió a generalizar.

Una de las cosas importantes al evaluar modelos que son de clasificaciones binarias es que existe algo llamado “Matriz de

Confusión” que nos permitirá visual como fueron las predicciones del modelo.

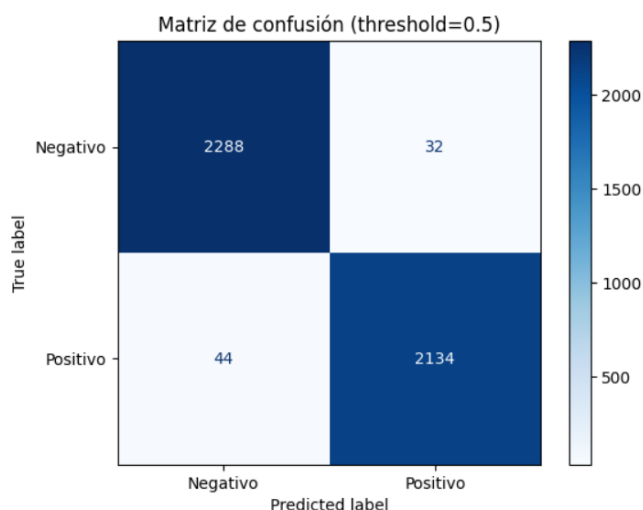


Fig. 7. Matriz de confusión para dataset de test

Como se observa en la Fig. 7, se puede ver que el modelo tiene un muy buen desempeño ya que la mayoría de sus predicciones positivas son correctas y sus predicciones negativas también lo son.

XVI. PREDICCIONES PARA TEXTO NUEVO

La última prueba que podemos hacer para saber si el modelo es un buen modelo y cumple con las necesidades es buscar hacer predicciones en texto crudo, es decir texto que escribamos nosotros, busquemos en internet o similar.

Negativo	3	4
Positivo	5	4
True label/ Predicted label	Negativo	Positivo

XVII. CONCLUSIONES

En esta primera entrega se construyó una estructura sólida para el desarrollo de un modelo de detección de noticias falsas. Después de un proceso de limpieza, transformación y vectorización del dataset, se entrenó un modelo recurrente capaz de generalizar de manera adecuada, evidenciado por métricas consistentes en train, validation y test.

La arquitectura empleada demostró un desempeño satisfactorio y la implementación de funciones como Early Stopping y Checkpoint permitió asegurar estabilidad y evitar sobreajuste. Además, al crear un vocabulario propio y su reutilización en los notebooks posteriores establece una base replicable para futuros experimentos.

Aunque los resultados fueron positivos aún se necesitan mejoras, donde una de las más importantes es asegurar que la transformación de los textos crudos se está haciendo de la misma manera en la que se hizo para los datasets de entrenamiento. Con esto podrá ser posible incrementar la precisión del modelo, mejorar su capacidad de interpretación y robustez frente a variaciones sintácticas o semánticas en el texto crudo.

SEGUNDA PARTE, MEJORAS DEL MODELO

Para esta segunda parte, el objetivo es hacer mejoras al modelo para que se pueda generar un mejor desempeño en las predicciones.

Tomando en cuenta los resultados de la primera prueba del proyecto se decidieron hacer varios ajustes como cambios en el proyecto para buscar el mejor rendimiento del modelo.

XVIII. CAMBIO DE DATASET

Se encontró un dataset con más observaciones en la plataforma de Kaggle, la cual cuenta con 2 datasets que están divididos en fake news y true news. Se decidió cambiar de dataset para tener más observaciones y ver si esto ayuda a que el modelo generalice mejor y sean mejores las predicciones.

Lo primero que tenemos que hacer es nuevamente la limpieza y transformaciones necesarias para preparar los datos para el modelo. En este caso son 2 datasets iniciales:

- Fake news (23,481 observaciones, 5 atributos)
- True news (21,417 observaciones, 5 atributos)

Link de los datasets:

<https://www.kaggle.com/code/therealsampat/fake-news-detection/input>

```
df_fake["class"] = 0
df_true["class"] = 1
df_fake.shape, df_true.shape

df_fake = df_fake.sample(n=len(df_true),
random_state=42)
df_fake.shape, df_true.shape
```

Código 11. Configuraciones previas al merge

Como se puede ver en el Código 11, antes de concatenar ambos datasets, definimos la etiqueta de 0 (fake news) y 1 (true news) que servirán para las predicciones. Posteriormente vamos a eliminar observaciones del dataset de fake news para que ambos datasets tengan el mismo tamaño y al momento de concatenarlos se encuentren balanceados los datos.

XIX. LIMPIEZA DE DATOS

Como se mencionó es necesario hacer nuestra limpieza del dataset. Para la limpieza del dataset se hicieron varias transformaciones similares a las que se hicieron en la parte anterior.

XIX.I Null count

Primero se revisa si el dataset tiene algún valor nulo que al momento de manipular los datos provoque problemas.

XIX.II Remove null

En caso de que haya valores nulos, estos se deben remover. En caso de este dataset no existieron valores nulos, pero de igual forma se eliminó cualquier valor que pudiera ser nulo.

XIX.II Remove columns (text, subject, date)

Una vez que se limpia el dataset, se deben quitar las columnas (features) que no serán relevantes para el entrenamiento ni para las predicciones. En este caso las columnas que fueron removidas fueron las siguientes:

- text: en este caso se decidió eliminar este atributo debido a que ya se tiene el título para este modelo
- subject: no utilizaremos el género de la noticia para las predicciones.
- Date: no se utilizará la fecha para las predicciones

Como se puede observar para esta mejora se decidió optar por utilizar el título de la noticia en vez de texto completo por varias razones:

- Al ser menos texto, el modelo se entrena más rápido y se pueden identificar patrones más fáciles.
- A veces cuando una persona logra identificar si una noticia es falsa o no, es a través de la redacción de los títulos en vez del texto completo.

XV.VI Hacer shuffle de datos

Se realiza un shuffle al dataset ya que es necesario mezclar para que el modelo no entrene a partir de algunas noticias y al hacer la pruebas sea con noticias diferentes, sino que el modelo pueda ver diferentes casos de noticias, pero no todas las noticias.

XV.VII Eliminar caracteres especiales y poner en minúsculas

Finalmente, lo que debemos hacer para los datasets es eliminar caracteres especiales y convertir todos los textos a minúsculas, de esta forma nos asegurarnos que palabras iguales no se clasifiquen en tokens diferentes por tener mayúscula o carácter especial.

Una vez finalizada la limpieza de los datos, en este caso el dataset termina con las siguientes dimensiones:

42,834 rows x 2 columns

Ecuación. 1. Dimensiones del dataset después de limpieza

XX. TRANSFORMACIÓN DE LOS DATOS

Antes de poner el dataset dentro del modelo se debe separar el dataset en train, val y test.

Para este dataset se hizo una división en una proporción de 70/15/15 donde habrá 29,983 muestras para entrenamiento (70%), 6,425 muestras para validación (15%) y 6,426 muestras para prueba (15%), para que nuestro split siempre sea el mismo utilizamos una semilla de “random_state = 42”, de igual forma se define “stratify=df_shuffle[“class”]” para que al separar los 3 datasets se mantengan balanceados.

$$29,983 + 6,425 + 6,426 = 42,834$$

Ecuación. 3. Tamaño del dataset a trabajar

Una vez teniendo los datasets de entrenamiento, validación y prueba, el siguiente paso es hacer una última transformación a los datos para poder meterlos al modelo.

XX.I Tokenización y Vectorización

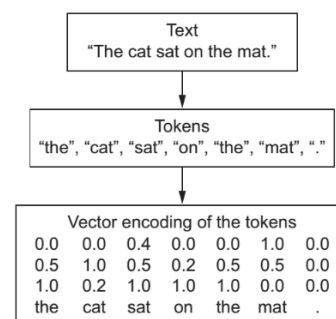


Fig. 8. Ejemplo de Tokenización y Vectorización.

Para este proceso se utilizó la capa de **TextVectorization** de Keras.

```
from tensorflow.keras.layers import TextVectorization
MAX_SEQUENCE_LENGTH = 15
VOCAB_SIZE = 10000

int_vectorize_layer = TextVectorization(
    max_tokens=VOCAB_SIZE,
    output_mode='int',
    output_sequence_length=MAX_SEQUENCE_LENGTH)
```

Código 12. Función de Tokenización y Vectorización

En el código 12, se puede ver una función que genera una matriz numérica de cada fila que representa un texto y cada columna un índice del vocabulario. Para mantener una longitud uniforme y truncar los textos que sean muy largos se definió un **MAX_SEQUENCE_LENGTH = 15**.

```
texts = dataset["text"].astype(str).values
labels = dataset["label"].values

BATCH_SIZE = 128
```

```
dataset = tf.data.Dataset.from_tensor_slices((texts,
labels)).batch(BATCH_SIZE)
```

Código 13. Separación de texto y etiquetas

En el código 13, se hace la separación del texto y las etiquetas de cada dataset para poder convertir los datasets a **tensorflow.data.Dataset** y agruparlos en batches (BATCH_SIZE = 128).

Para sacar el vocabulario solo se va a utilizar el dataset del train debido a que al momento de entrenar es importante solo tomar en cuenta la información del dataset del train para que al probar el modelo en validation y test este pueda predecir sin haber visto las palabras de los nuevos datasets.

```
train_text = train_ds.map(lambda text, label: text)
int_vectorize_layer.adapt(train_text)
```

```
def int_vectorize_text(text, label):
    text = tf.expand_dims(text, -1)
    return int_vectorize_layer(text), label
```

```
dataset = dataset.map(int_vectorize_text)
```

Código 14. Vectorizar y transformar datasets

Como se muestra en el Código 14, una vez teniendo los tokens vectorizados esto se mapea a cada uno de los datasets para convertir todos los tokens de los textos a los vectores (números enteros) a partir de la vectorización (vocabulario) que se generó del dataset de train.

```
AUTOTUNE = tf.data.AUTOTUNE
```

```
def configure_dataset(dataset):
    return dataset.cache().prefetch(buffer_size=AUTOTUNE)
```

```
int_dataset = configure_dataset(dataset)
```

Código 15. Configuración del dataset

Como se muestra en el Código 15, con esto se mantendrán los datos en memoria y prepara los datos del siguiente lote mientras se entrena el actual.

```
vocab = int_vectorize_layer.get_vocabulary()
```

Código 16. Vocabulario

De igual forma para poder identificar patrones en los textos es necesario tener un vocabulario, para esto el vocabulario nuevamente se sacó del dataset de train donde lo que se hace es contar cuantas veces los tokens son repetidos y se acomodan de mayor frecuencia a menor frecuencia para tomar los tokens que son usados de manera más usual en fake news y en noticias reales.

XXI. IMPORTACIÓN DE DATASETS Y VOCABULARIO

Una vez teniendo los datasets preparados se exportan al igual que el vocabulario para que así se puedan importar a cualquier otro notebook que requiera esos archivos.

XXII. ENTORNO PARA TRABAJAR EN NOTEBOOK ENT

Para poder empezar el entrenamiento es necesario importar los datasets de train y validation y el vocabulario.

XXIII. PREPARACIÓN DEL DATASET DE TRAIN

Antes de crear y entrenar el modelo es necesario revisar el dataset de train y validation para saber qué es lo que contienen los archivos que estamos importando.

De igual forma importamos el vocabulario que servirá para el entrenamiento.

XXIV. VERSIONES DE ARQUITECTURA DEL MODELO

Otra de las configuraciones que realizamos para la mejora del modelo fue la creación de una arquitectura más robusta para el modelo. Debido a que hacer configuraciones de la arquitectura una sola vez no nos ayuda a determinar que esa nueva arquitectura es la mejor se hicieron 3 nuevas arquitecturas con distintas configuraciones que permitirán evaluar que arquitectura es la más adecuada para este problema.

XXIV.1 Segunda versión

Para la segunda versión se decidió configurar de la siguiente manera:

- Embedding
- Bidirectional (Red neuronal recurrente para generar memoria, en este caso se configuro para permitir que la red vea toda la secuencia)
- Capas densas

Métodos de regularización como:

- SpatialDropout1D (apagan las palabras completas de manera temporal)
- GlobalMaxPooling1D (capturar la presencia de patrones relevantes en cualquier parte del texto)
- BatchNormalization (normalizar las activaciones de cada capa para mantener una media y desviación estándar consistente)
- Dropout

```
model2 = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=len(encoder.get_vocabulary()),
                              output_dim=64, mask_zero=True),
    tf.keras.layers.SpatialDropout1D(0.2),
    tf.keras.layers.Bidirectional([tf.keras.layers.LSTM(64,
return_sequences=True, dropout=0.1,
recurrent_dropout=0.1)),
    tf.keras.layers.GlobalMaxPooling1D(),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

Código 17. Segunda versión de la arquitectura

En esta versión permanecieron las mismas que en la primera versión del modelo teniendo las siguientes métricas:

- Binary cross-entropy
- Optimizer Adam
- Accuracy

Y también funciones como:

- Early stop
- Checkpoint

Mantenemos la definición del entrenamiento igual que la primera versión.

```
val_loss, val_acc = model2.evaluate(val_dataset)
print('Val Loss:', val_loss)
print('Val Accuracy:', val_acc)

... 51/51 ————— 1s 23ms/step - accuracy: 0.9618 - loss: 0.1145
Val Loss: 0.1190527081489563
Val Accuracy: 0.9614008069038391
```

Fig. 9. Resultados del entrenamiento con validation

En la Fig. 9 se puede observar que el resultado del dataset de validation.

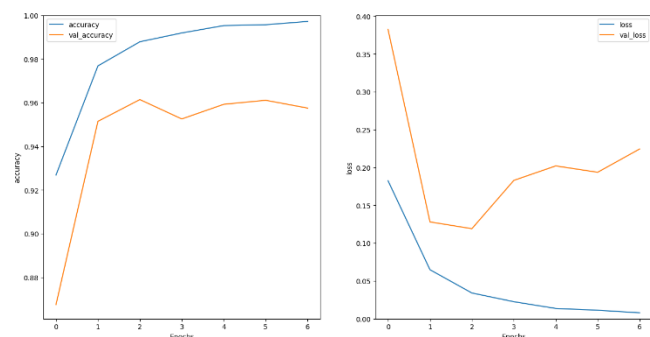


Fig. 10. Métricas Train vs Validation

En la Fig.10 se observa la comparación de las métricas de accuracy y loss del train y de validation.

Como se mencionó antes al evaluar modelos que son de clasificaciones binarias existe la “Matriz de Confusión”.

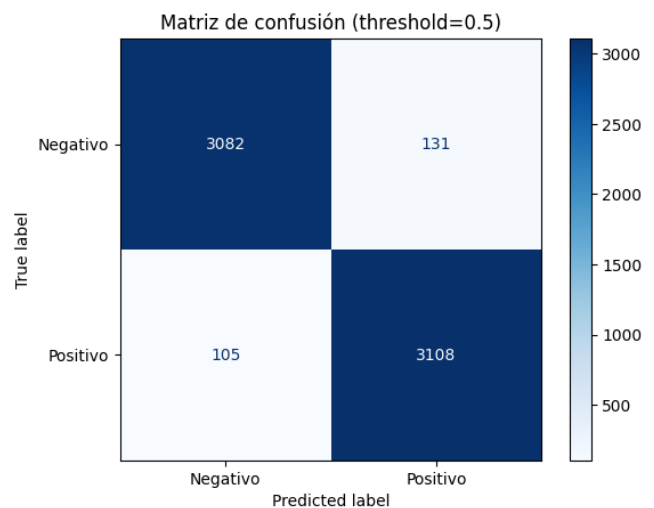


Fig. 11. Matriz de confusión para dataset de test

Como se observa en la Fig. 7, se puede ver que el modelo tiene buen desempeño, sin embargo, su rendimiento es menor que el de la primera versión.

La última prueba es buscar hacer predicciones en texto crudo, es decir texto que escribamos nosotros, busquemos en internet o similar.

Negativo	13	10
Positivo	8	12
True label/ Predicted label	Negativo	Positivo

XXIV.II Tercera versión

Para la segunda versión se decidió configurar de la siguiente manera:

- Embedding
- Bidirectional (Red neuronal recurrente para generar memoria, en este caso se configuro para permitir que la red vea toda la secuencia)
- Capas densas

Métodos de regularización como:

- GlobalMaxPooling1D
- Dropout

```
model2 = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=len(encoder.get_vocabulary()), output_dim=128, mask_zero=True),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64, return_sequences=True, dropout=0.2, recurrent_dropout=0.2, kernel_regularizer=regularizers.l2(1e-4))),
    tf.keras.layers.GlobalMaxPooling1D(),
    tf.keras.layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(1e-4)),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

Código 18. Segunda versión de la arquitectura

En esta versión permanecieron las mismas que en la primera versión del modelo teniendo las siguientes métricas:

- Binary cross-entropy
- Optimizer Adam
- Accuracy

Y agregando una función adicional a las de la primera versión:

- Early stop
- Checkpoint
- Reduce LR (reduce la tasa de aprendizaje cuando la pérdida deja de mejorar)

Mantenemos la definición del entrenamiento igual que la primera versión.

```
val_loss, val_acc = model2.evaluate(val_dataset)

print('Val Loss:', val_loss)
print('Val Accuracy:', val_acc)
```

... 51/51 1s 22ms/step - accuracy: 0.9633 - loss: 0.1132
 Val Loss: 0.11897685378789902
 Val Accuracy: 0.9612451195716858

Fig. 12. Resultados del entrenamiento con validation

En la Fig. 12 se puede observar que el resultado del dataset de validation.

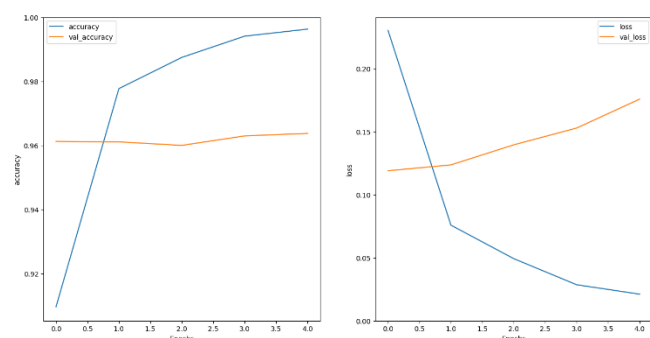


Fig. 13. Métricas Train vs Validation

En la Fig.13 se observa la comparación de las métricas de accuracy y loss del train y de validation.

Como se mencionó antes al evaluar modelos que son de clasificaciones binarias existe la “Matriz de Confusión”.

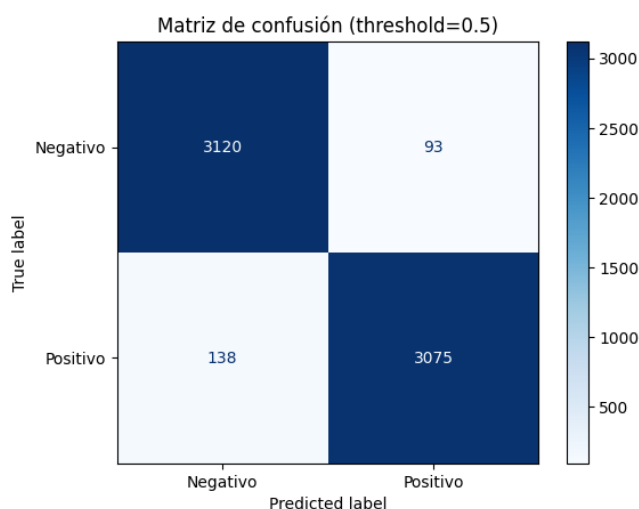


Fig. 14. Matriz de confusión para dataset de test

Como se observa en la Fig. 14, se puede ver que en este modelo en las predicciones atinadas subieron nuevamente, sin embargo, su rendimiento es menor que el de la primera versión.

La última prueba es buscar hacer predicciones en texto crudo, es decir texto que escribamos nosotros, busquemos en internet o similar.

Negativo	15	8
Positivo	8	12
True label/ Predicted label	Negativo	Positivo

XXV. ARQUITECTURA VERSIÓN FINAL DEL MODELO

Evaluando las tres versiones previas se decidió buscar lo mejor de cada arquitectura o lo que pudiera ayudar a sacar características importantes de la información y tener más precisión para generar una versión final del modelo además de tomar en cuenta cuales fueron sus resultados en la matriz de confusión y en las predicciones de los textos crudos. Esta fue la versión que proporcionó mejores resultados en entrenamiento, validación, prueba y textos crudos, que se configuro de la siguiente manera:

- Embedding (transformación de vectores a valores que pueda ajustar el modelo),
- 2 Bidirectional (Red neuronal recurrente para generar memoria, en este caso se configuro para permitir que la red vea toda la secuencia y vuelva a generar memoria)
- Capas densas (con funciones de activación)

Métodos de regularización como:

- SpatialDropout1D (apagan las palabras completas de manera temporal)
- GlobalMaxPooling1D (capturar la presencia de patrones relevantes en cualquier parte del texto)

- BatchNormalization (normalizar las activaciones de cada capa para mantener una media y desviación estándar consistente)
- Dropout (apagan neuronas de manera temporal),

```
from tensorflow.keras import regularizers

model2 = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=len(encoder.get_vocabulary()), output_dim=128, mask_zero=True),
    tf.keras.layers.SpatialDropout1D(0.2),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64, return_sequences=True, dropout=0.2, recurrent_dropout=0.2, kernel_regularizer=regularizers.l2(1e-4))),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(32, return_sequences=True, dropout=0.2, recurrent_dropout=0.2, kernel_regularizer=regularizers.l2(1e-4))),
    tf.keras.layers.GlobalMaxPooling1D(),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(1e-4)),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

Código 19. Arquitectura del modelo

En el código 19, se puede ver cómo es la estructurada de la arquitectura de igual forma se tomó en cuenta que la última capa de salida debe tener una función de activación “sigmoid” debido a que este problema es de clasificación binaria. Considerando esa arquitectura se espera que tenga el mejor rendimiento comparado con las demás arquitecturas probadas.

XXVI. CONFIGURACIONES PARA EL MODELO DURANTE ENTRENAMIENTO

De igual manera se deben configurar las métricas de desempeño y de ajuste y funciones regularizadoras para el modelo.

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

model2.compile(
    loss='binary_crossentropy',
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    metrics=['accuracy',
            tf.keras.metrics.Precision(name='precision'),
            tf.keras.metrics.Recall(name='recall'),
            tf.keras.metrics.AUC(name='auc')]
)

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True,
    verbose=1
)

checkpoint = ModelCheckpoint(
    filepath=model_path,
    monitor='val_loss',
    save_best_only=True,
    save_weights_only=False,
    mode='min',
    verbose=1
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=2,
    min_lr=1e-6,
    verbose=1
)
```

Código 20. Inicialización de parámetros nuevos

Como se puede ver en el Código 20, aquí definimos que métricas vamos a usar para medir el desempeño del entrenamiento donde también se puede ver que agregamos métricas adicionales para la evaluación del desempeño.

- Binary cross-entropy: que sirve para evaluar el error/perdida de las predicciones al momento del entrenamiento.
- Optimizer Adam: esto sirve para ajustar el learning rate dependiendo del desempeño del entrenamiento.
- Accuracy: esto sirve para saber cuánta precisión tuvo el modelo durante el entrenamiento.
- Precisión: esto sirve para medir qué proporción de las predicciones positivas fueron correctas
- Recall: este sirve para medir qué proporción de los positivos reales sí se detectaron.
- Auc: este sirve para medir qué tan bien el modelo separa clases en general.

Y definimos también funciones como:

- Early stop: detener el entrenamiento cuando no encuentra una mejora considerable en, este caso, la pérdida de validation.
- Checkpoint: Guardar los pesos del modelo cuando este mejore a partir de sus métricas.
- Reduce LR: (reduce la tasa de aprendizaje cuando la pérdida deja de mejorar)

XXVII. RESULTADO DE ENTRENAMIENTO

Ahora definimos el entrenamiento del modelo, en este caso una de las cosas que se pueden hacer para tener mejores predicciones es penalizar al modelo por equivocarse en ciertas predicciones, esto se va a implementar de manera que se penalizara más al modelo si se equivoca en una fake news que en una noticia real, esto con el objetivo de que pueda detectar mejor las fake news.

```
class_weight = {
    0: 1.0, # real
    1: 1.5 # fake (más importante)
}

history = model2.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=20,
    callbacks=[early_stop, checkpoint, reduce_lr],
    class_weight=class_weight
)
```

Código 21. Arquitectura del modelo

Como se ve en el Código 21, para el entrenamiento del modelo hacemos las siguientes configuraciones:

- Dataset con el que va a entrenar
- Numero de epochs
- Con que dataset va a validar

- ```

root@localhost:~/lib/python.12/dist-packages/keras/src/layers/layer.py:965: UserWarning: Layer 'global_max_pooling2d' of type GlobalMaxPooling2D is not supported on the GPU. Falling back to the CPU implementation.
warnings.warn('')

235/235 0s 260ms/step - accuracy: 0.8755 - auc: 0.9413 - loss: 0.3886 - precision: 0.8401 - recall: 0.9273
Epoch 1: val_loss improved from inf to 2.6379, saving model to /content/gdrive/MyDrive/ESQUELA/1RS/710/2-2/Modelo-2.2/EVIDENCIA
235/235 0s 260ms/step - accuracy: 0.8755 - auc: 0.9415 - loss: 0.3880 - precision: 0.8404 - recall: 0.9321
Epoch 2/20

235/235 0s 255ms/step - accuracy: 0.9714 - auc: 0.9953 - loss: 0.1195 - precision: 0.9619 - recall: 0.9811
Epoch 3: val_loss improved from 2.6379 to 0.12105, saving model to /content/gdrive/MyDrive/ESQUELA/1RS/710/2-2/Modelo-2.2/EVID
235/235 0s 267ms/step - accuracy: 0.9714 - auc: 0.9953 - loss: 0.1194 - precision: 0.9619 - recall: 0.9811
Epoch 3/20

235/235 0s 254ms/step - accuracy: 0.9846 - auc: 0.9982 - loss: 0.0725 - precision: 0.9787 - recall: 0.9904
Epoch 4: val_loss did not improve from 0.12105
235/235 0s 262ms/step - accuracy: 0.9846 - auc: 0.9982 - loss: 0.0725 - precision: 0.9787 - recall: 0.9904
Epoch 4/20

235/235 0s 259ms/step - accuracy: 0.9903 - auc: 0.9990 - loss: 0.0545 - precision: 0.9871 - recall: 0.9934
Epoch 4: val_loss did not improve from 0.12105

Epoch 4: ReduceLROnPlateau reducing learning rate to 0.00050000000237487275.
235/235 0s 267ms/step - accuracy: 0.9903 - auc: 0.9990 - loss: 0.0545 - precision: 0.9871 - recall: 0.9934
Epoch 5/20

235/235 0s 261ms/step - accuracy: 0.9929 - auc: 0.9994 - loss: 0.0429 - precision: 0.9916 - recall: 0.9941
Epoch 5: val_loss did not improve from 0.12105
235/235 0s 263ms/step - accuracy: 0.9929 - auc: 0.9994 - loss: 0.0428 - precision: 0.9916 - recall: 0.9941
Epoch 6/20

235/235 0s 261ms/step - accuracy: 0.9968 - auc: 0.9998 - loss: 0.0287 - precision: 0.9960 - recall: 0.9976
Epoch 6: val_loss did not improve from 0.12105

Epoch 6: ReduceLROnPlateau reducing learning rate to 0.00025000000118743628.
235/235 0s 270ms/step - accuracy: 0.9968 - auc: 0.9998 - loss: 0.0287 - precision: 0.9960 - recall: 0.9976
Epoch 6: early stopping
Restoring model weights from the end of the best epoch: 2.

```

The figure consists of two line plots side-by-side, both sharing an x-axis labeled 'Epochs' ranging from 0 to 5.

The left plot shows accuracy on the y-axis, ranging from 0.93 to 1.00. It contains two lines: 'accuracy' (blue) and 'val\_accuracy' (orange). The training accuracy starts at approximately 0.929 and increases steadily to about 0.998. The validation accuracy starts at approximately 0.956, peaks at epoch 1 (0.961), and then fluctuates slightly, ending at approximately 0.963.

The right plot shows loss on the y-axis, ranging from 0.00 to 0.25. It contains two lines: 'loss' (blue) and 'val\_loss' (orange). The training loss starts at approximately 0.250 and decreases steadily to about 0.025. The validation loss starts at approximately 0.275, drops to epoch 1 (0.125), and then generally increases, ending at approximately 0.202.

| Epochs | accuracy | val_accuracy | loss  | val_loss |
|--------|----------|--------------|-------|----------|
| 0      | 0.929    | 0.956        | 0.250 | 0.275    |
| 1      | 0.976    | 0.961        | 0.110 | 0.125    |
| 2      | 0.987    | 0.959        | 0.068 | 0.152    |
| 3      | 0.992    | 0.960        | 0.052 | 0.181    |
| 4      | 0.996    | 0.959        | 0.035 | 0.181    |
| 5      | 0.998    | 0.963        | 0.025 | 0.202    |

```

▶ val_loss, val_acc, val_precision, val_recall, val_auc = model2.evaluate(val_dataset)

print('Val Loss:', val_loss)
print('Val Accuracy:', val_acc)
print('Val Precision:', val_precision)
print('Val Recall:', val_recall)
print('Val AUC:', val_auc)

```

---

```

... 51/51 ————— 2s 38ms/step - accuracy: 0.9621 - auc: 0.9939 - loss: 0.1184
Val Loss: 0.12105006724596024
Val Accuracy: 0.9614008069038391
Val Precision: 0.9634146094322205
Val Recall: 0.9592154622077942
Val AUC: 0.9935857057514111

```

| Layer (type)                              | Output Shape    | Param #   |
|-------------------------------------------|-----------------|-----------|
| embedding (Embedding)                     | (None, 15, 128) | 1,280,000 |
| spatial_dropout1d (SpatialDropout1D)      | (None, 15, 128) | 0         |
| bidirectional (Bidirectional)             | (None, 15, 128) | 98,816    |
| bidirectional_1 (Bidirectional)           | (None, 15, 64)  | 41,216    |
| global_max_pooling1d (GlobalMaxPooling1D) | (None, 64)      | 0         |
| batch_normalization (BatchNormalization)  | (None, 64)      | 256       |
| dense (Dense)                             | (None, 64)      | 4,160     |
| dropout (Dropout)                         | (None, 64)      | 0         |
| dense_1 (Dense)                           | (None, 1)       | 65        |

Nuevamente checamos que tipo de modelo se está importando y verificamos que sea el que se quiere utilizar, como se puede mostrar en la Fig. 18.

## XXIX. PREDICCIONES PARA DATASET DE TRAIN

Como se mencionó antes, para saber si el modelo está funcionando de manera adecuado es necesario también hacer pruebas con un dataset específico para pruebas, es decir, observaciones que el modelo nunca haya visto antes.

TN=3091 FP=122 FN=123 TP=3090  
Accuracy=0.9619 Precision=0.9620 Recall=0.9617 F1=0.9619 ROC-AUC=0.9936

Reporte de clasificación:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Negativo     | 0.96      | 0.96   | 0.96     | 3213    |
| Positivo     | 0.96      | 0.96   | 0.96     | 3213    |
| accuracy     |           |        | 0.96     | 6426    |
| macro avg    | 0.96      | 0.96   | 0.96     | 6426    |
| weighted avg | 0.96      | 0.96   | 0.96     | 6426    |

Fig. 19. Resultados del entrenamiento con test

En la Fig. 19 se puede observar que el modelo tuvo un buen desempeño lo que nos da a entender que durante el entrenamiento el modelo si aprendió a generalizar y a tener una buena precisión en las predicciones.

De igual forma que en la primera versión para evaluar modelos que son de clasificaciones binarias usamos la “Matriz de Confusión” que nos permitirá visual como fueron las predicciones del modelo.

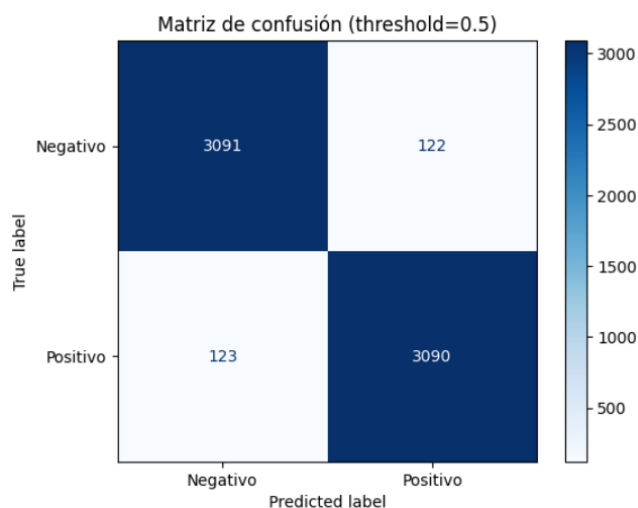


Fig. 20. Matriz de confusión para dataset de test

Como se observa en la Fig. 20, se puede ver que el modelo tiene un buen desempeño ya que la mayoría de sus predicciones positivas son correctas y sus predicciones negativas también lo son. A pesar de que estos resultados de la matriz de confusión son más bajos que la primera versión del modelo, debemos también observar como se comportan las predicciones en textos crudos que eso fue lo que mas falló el primer modelo. Con esta matriz podemos comprobar que aun teniendo errores el modelo no memorizo el set de

entrenamiento y que aprendió a generalizar con una perdida bastante baja considerando el número de observaciones totales que tenemos.

## XXX. PREDICCIONES PARA TEXTO NUEVO

La última prueba, como se mencionó, es buscar hacer predicciones en texto crudo, es decir texto que escribamos nosotros, busquemos en internet o similar.

|                             |          |          |
|-----------------------------|----------|----------|
| Negativo                    | 14       | 9        |
| Positivo                    | 5        | 15       |
| True label/ Predicted label | Negativo | Positivo |

## XXXI. CONCLUSIONES

Para esta segunda parte del proyecto se decidió por una versión final del modelo de detección de noticias falsas que lograra un buen ajuste en las métricas de entrenamiento, validación y prueba, pero también mostrara un desempeño significativo al usar con textos crudos. Después de varias iteraciones sobre la arquitectura inicial, la combinación de capas Embedding, redes LSTM bidireccionales, funciones de regularización (Dropout, SpatialDropout1D, BatchNormalization, penalización L2) y el uso de class weights se pudo tener un modelo final más robusto, con buena capacidad de generalización y menor riesgo de sobreajuste. Esto muestra que el proceso de limpieza, transformación, vectorización y entrenamiento fue adecuado para aprender patrones útiles para este proyecto.

Un cambio importante en esta mejora fue el uso exclusivo de los títulos de las noticias en lugar del texto completo. Esto debido a que, en muchos casos, el título concentra gran parte del lenguaje sensacionalista o llamativo que suele utilizarse para atraer al público en noticias falsas, siendo esto una señal fuerte para el modelo. También el hecho de trabajar con menos texto logro hacer más eficiente el proceso de entrenamiento y facilito la identificación de patrones recurrentes en los títulos de las noticias. Los resultados obtenidos nos ayudan a comprobar que el cambio de atributo, bajo estas condiciones, logra que el modelo sea capaz de distinguir de forma entre fake news y noticias reales.

Aun así, es importante saber que modelos como este no pueden considerarse perfectos ni definitivos, ya que solo se logra ver una parte muy específica de la noticia: el título y su etiqueta (fake news o noticia real). La credibilidad de una noticia también depende de otros factores que no se incluyen en este proyecto, como el medio en el que se publica, la reputación de la fuente o el autor, la fecha y el contexto de la información que tiene la noticia, la existencia de otras noticias que ayuden a confirmar o contradecir el mismo hecho o acontecimiento, etc. Por ello, este modelo puede usarse como una herramienta de

apoyo que ayude a filtrar y priorizar posibles fake news a partir de sus títulos, pero no como un sustituto del análisis contextual ni de la verificación con múltiples fuentes. Una de las cosas que se pudieran implementar para la mejora del modelo sería integrar información adicional del contenido de las noticias y de su contexto (metadatos, comportamiento en redes, contraste con otras fuentes), para así hacer un modelo o herramienta más robusta frente a nuevas formas de desinformación.

## XXXII. REFERENCIAS

- *Pulkit17/Fake-News-Detection-dataset* · *Datasets at Hugging Face*. (s. f.). <https://huggingface.co/datasets/Pulkit17/Fake-News-Detection-dataset>
- TensorFlow Developers. (2024). *TextVectorization Layer*. [https://www.tensorflow.org/api\\_docs/python/tf/keras/layers/TextVectorization](https://www.tensorflow.org/api_docs/python/tf/keras/layers/TextVectorization)
- KPMG. (2024). *Clues to help identify fake news*. [https://kpmg.com/xx/en/our-insights/ai-and-technology/clues-to-help-identify-fake-news.html?utm\\_source](https://kpmg.com/xx/en/our-insights/ai-and-technology/clues-to-help-identify-fake-news.html?utm_source)
- Cornell University Library. (2020). *How to spot fake news* [Infographic] [https://guides.library.cornell.edu/evaluate\\_news/infographic?utm\\_source](https://guides.library.cornell.edu/evaluate_news/infographic?utm_source)